

Machine Learning Driven Playlists-scalable & coherent

SioK Fei Lei • 06.12.2025

Problem Statement

- Playlist creation doesn't scale, manually done
- Growing customer base increases demand
- Need consistent, high-quality playlists

Objective

- Evaluate if Machine Learning can support Moosic's music experts
- Evaluate whether clusters “belong together

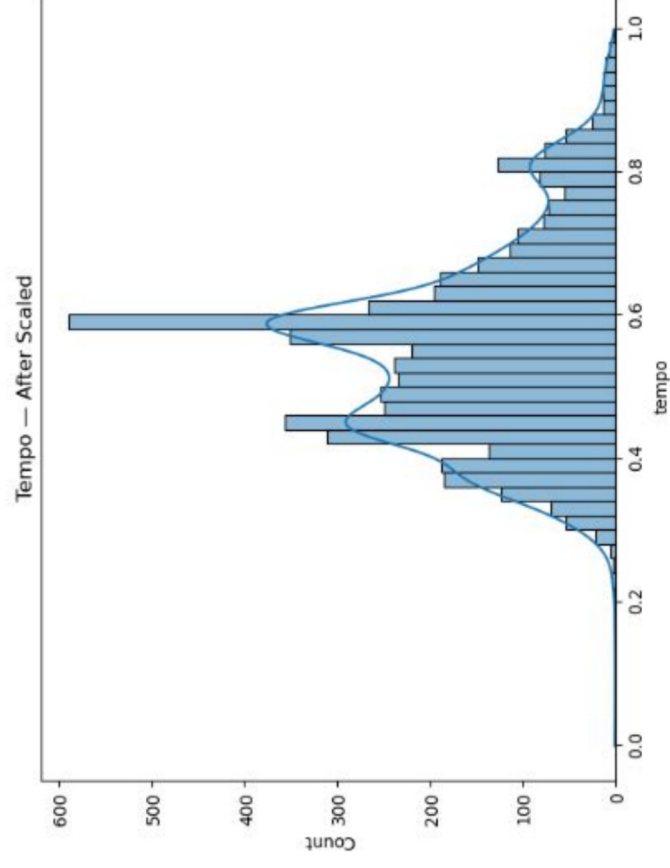
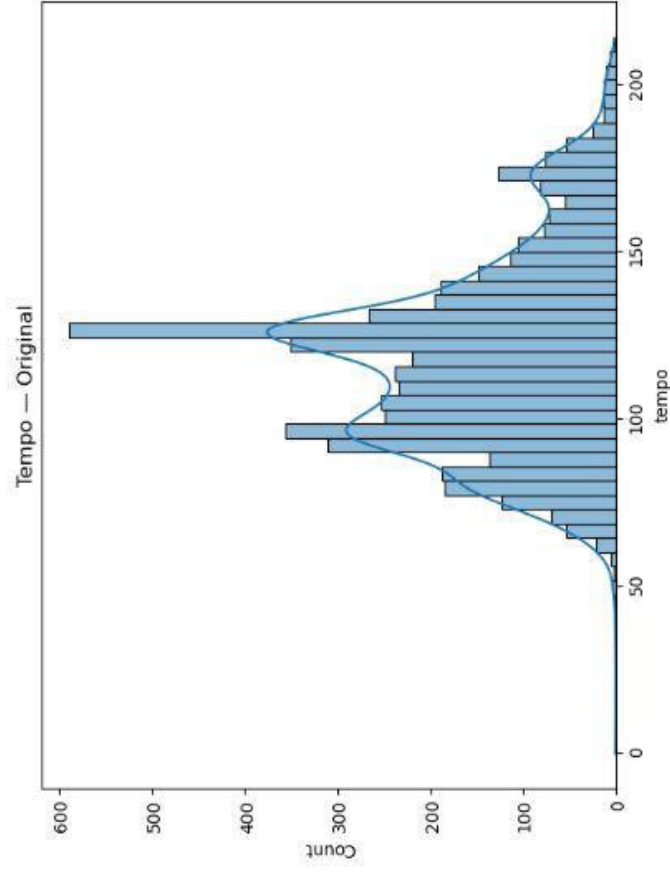
— — —

- ## Why: Reduce noise & simplify model.

tempo	110.376	125.039	93.698
--------------	---------	---------	--------

Standardizing Audio Features

- **Scaling:** MinMaxScaler - prevents features from dominating the clustering



How many components to keep (using PCA)

--- --

- Used **PCA** to reduce dimensionality
- Applied **cumulative explained variance** method
- Selected **6 principal components**
- **95% variance threshold** (industry standard in audio analysis)

KMeans Clustering Method_How Many Playlists?

- **Goal:** find the optimal number of playlists for songs.
- Initially set a max. Of **30** clusters, (manageable number)

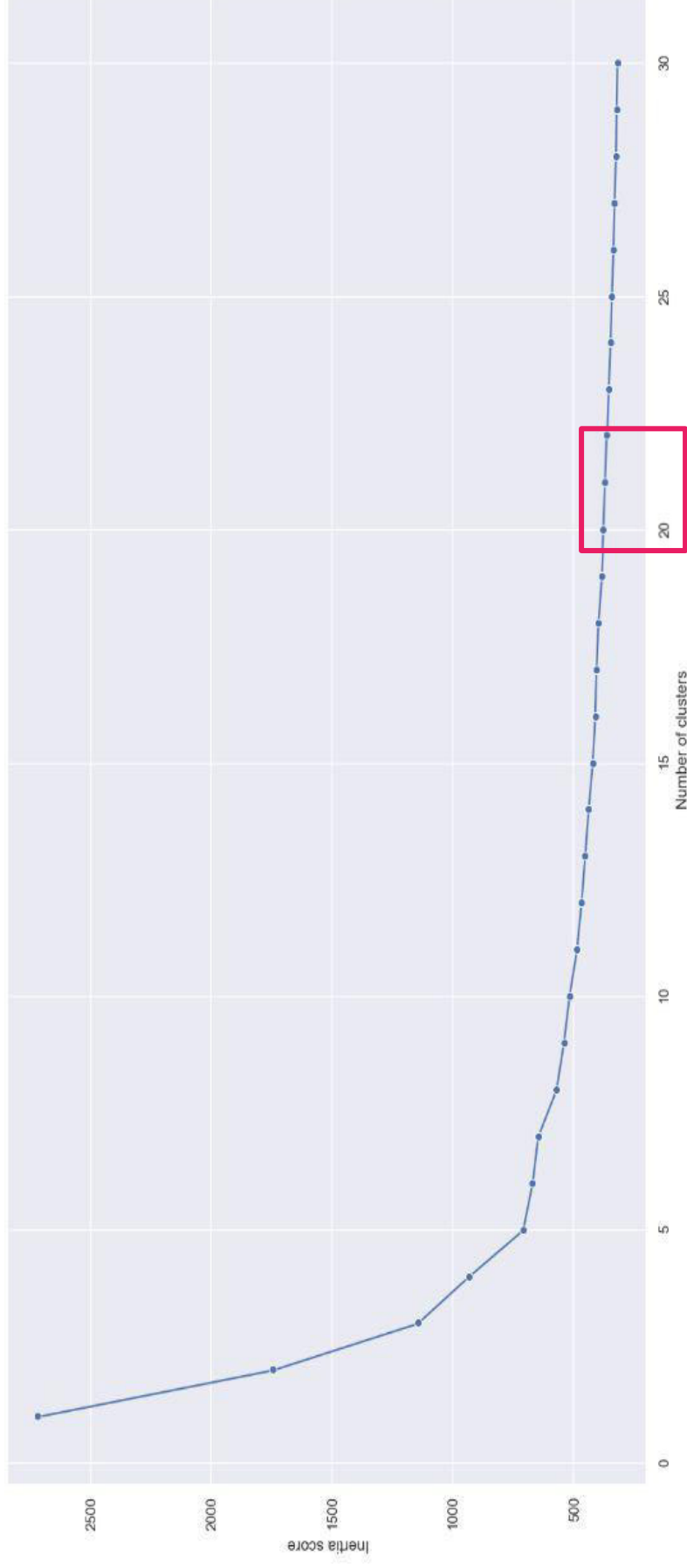
Methods used:

1st evaluation: Elbow method (inertia), resulting **~20-22 clusters**



Validation: Silhouette score, resulting **~20-22 clusters**

Clustering-Elbow method



Clustering-Silhouette score



Business view: How Many Playlists Make Sense for Moosic?

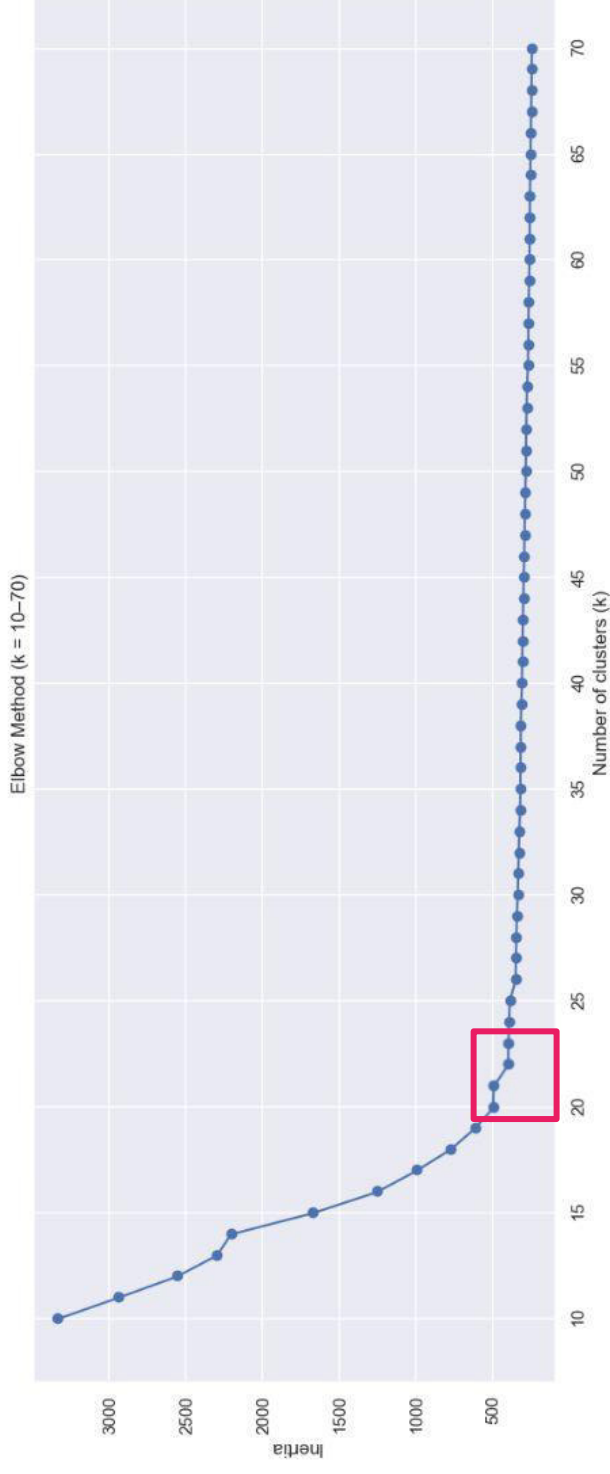
Moosic's aims:

- Each playlist shall feel unique, individual and marketable

Refined research by **testing cluster sizes 10 to 70 using elbow method:**

- **~20-22 clusters** consistently resulted in findings

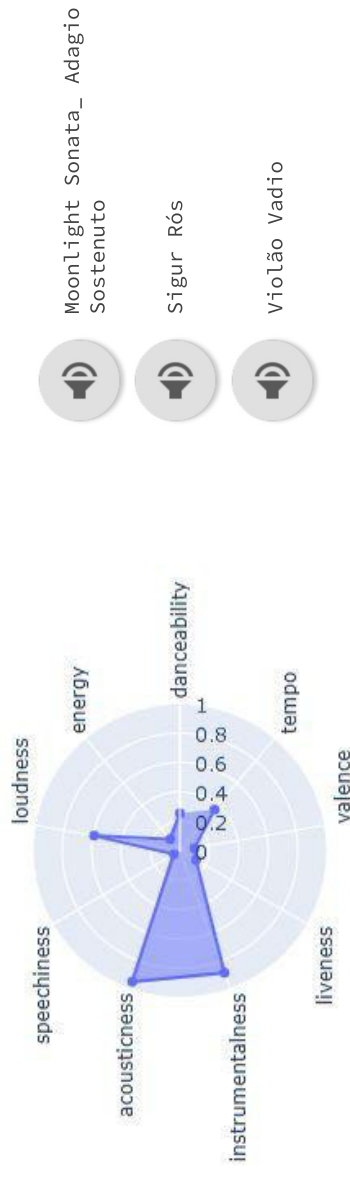
Findings of Clustering-Elbow method



Conclusion: ~20–22 clusters give the best balance for business usability and quality.

RESULTS: Playlist Insights-*compute KMeans using k=21 clusters*

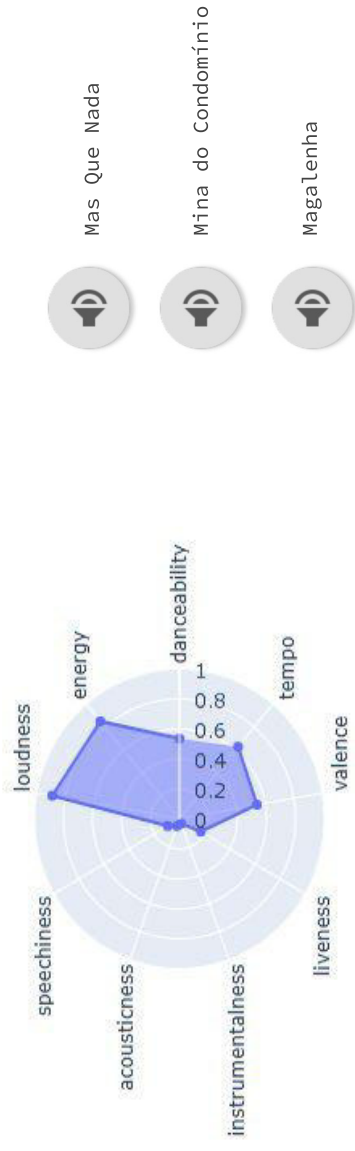
Cluster 0 feature profile



- High acousticness & instrumentalness: A mix of soft piano pieces and quiet acoustic songs including modern classical composers.
- Playlist for peaceful, calm moods suitable for minimalist, relaxation and reflection

RESULTS: Playlist Insights

Cluster 4 feature profile



- High loudness and energy giving danceability and positive moods with Brazilian, samba, and Latin vibes.
- Suitable for festivals, celebration and dances

Playlist Insights - summary

- The playlists in general feels cohesive and are created automatically driven by Machine Learning

Conclusion

Findings: Can Machine Learning Create Playlists for Moosic?

1. Machine Learning generated playlists are feasible and scalable

- Clustering produced **cohesive groups of songs** that “sound like they belong together”, ML supports Moosic with **scalable** way to curate playlists (automated)

2. 20–22 clusters give the best balance

- Playlists with distinct profiles with mixture of cluster sizes:
 - **Large:** 300–450 songs
 - **Medium:** 150–260
 - **Small:** 60–120
- This supports both **broad playlists** and **niche experiences**, potentially targeting different customers.

3. Business value

- A manageable number of playlists for Moosic
- Playlist sizes are **Large** enough to support future customer segments and scaling.

4. Technical takeaway

- **K-Means** worked well once audio features were standardized.

Limitations & Recommendations

1. Audio features are not enough for personalization

- **personal taste is not captured.**
- playlists are cohesive because there are sounds similarities, but **not yet customized** for individual listeners. Future prototypes should possibly include **mood or emotion labels, song usage context, Genres**

2. Recommended to combine Machine Learning with Moosic's music expertise

- can group songs efficiently. Best results come from a **human and machine learning approach**.

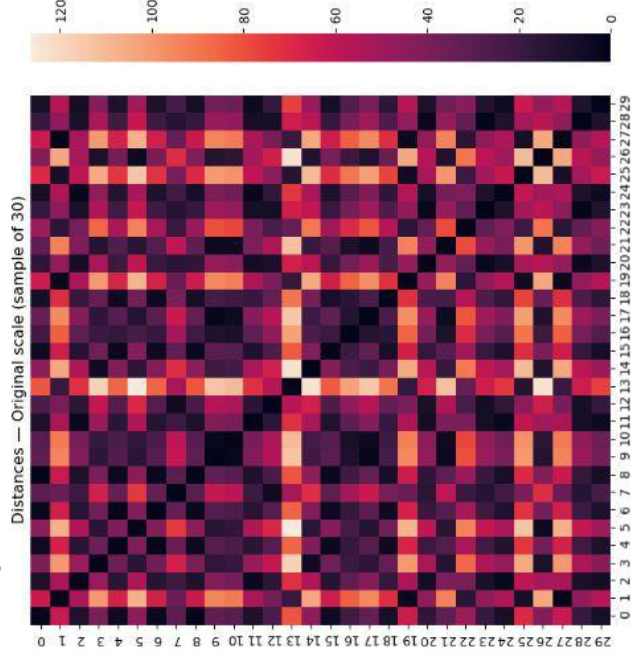
3. Need customer feedback and behavioural data analysis to enable personalized, user-specific playlists in future versions.

4. KMeans is good to support building the first prototype, possibility to use different ML models additionally to capture more underlying musical similarities

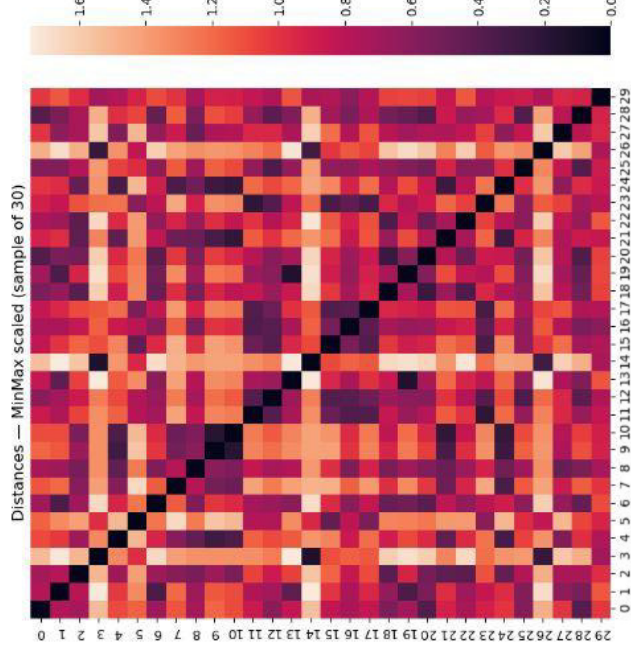
Thank you

Standardizing Audio Features

- **Scaling: MinMaxScaler** - prevents features from dominating the clustering



Inconsistent distances, dominated by features with large ranges



Distances are uniformed
Clustering becomes more fair and valid